

CLASSIC SNAKE GAME IN C

Design and Implementation of a Modular Console-Based Game

Md. Arik Ahnaf – 262 1388 042

Md. Faiyaz Farhan Adit – 262 4169 642

Jannatul Tufa Rodela – 262 2542 042

Mohammad Farhan Sadik – 262 5442 042

Abstract—This project presents the design and implementation of a console-based Classic Snake Game developed using the C programming language. The primary objective of the project is to apply fundamental programming concepts learned in CSE115 through the development of a complete interactive application. The game allows the player to control a continuously moving snake using the W, A, S, and D keys while collecting randomly generated food and avoiding collisions with the game boundaries and the snake's own body. The program follows a modular design in which separate source and header files are used for snake movement, food generation, keyboard input, board rendering, game-data management, and console utilities. Arrays are used to store the coordinates of individual snake segments, while a two-dimensional character array represents the game board. As food is collected, the snake grows and the player's score increases. The game also implements progressive difficulty by increasing the snake's movement speed at higher scores. In addition, the three highest scores are stored in a text file and retained between program executions. The completed project demonstrates the practical use of arrays, functions, loops, conditional statements, structures, file handling, random number generation, modular programming, and real-time keyboard input in C.

Keywords— Classic Snake Game, C programming, modular programming, collision detection, file handling.

I. INTRODUCTION

The Snake Game is a classic grid-based game in which the player controls a continuously moving snake with the objective of collecting food while avoiding collisions with the boundaries of the playing area and with the snake's own body. Although the basic gameplay is simple, implementing the game requires several programming components to operate together continuously, including user input, coordinate tracking, movement, collision detection, random food generation, score management, and screen rendering.

The purpose of developing this project was to apply the fundamental concepts introduced in CSE115 to a complete interactive program. Instead of using programming concepts separately in small exercises, the project combines arrays, loops, conditional statements, functions, structures, file handling, random number generation, and modular programming within a single application. The project was organized into multiple source and header files so that different responsibilities of the game could be implemented and maintained independently.

The developed Classic Snake Game provides real-time keyboard control using the W, A, S, and D keys, snake growth after food collection, wall and self-collision detection, score tracking, progressive speed increase, and persistent storage of the three highest scores. The game also includes a formatted console interface that displays the player's controls, current score, snake length, game status, and high-score information. Together, these features demonstrate how introductory C programming concepts can be integrated to build a structured and functional console-based game.

II. PROJECT OBJECTIVES AND SYSTEM DESIGN

A. Project Objectives

The primary objective of this project was to design and implement a complete Classic Snake Game using the C programming language while applying the programming concepts introduced in CSE115. The project was intended to move beyond isolated

programming exercises by combining multiple concepts into a single interactive application.

The specific objectives of the project were to develop a system that could provide responsive keyboard-based movement, represent and update the snake using coordinate arrays, generate food at valid random positions, increase the snake's length and score after food collection, detect collisions with both the game boundaries and the snake's own body, and progressively increase the difficulty of the game as the score rises. Another objective was to preserve the highest scores between program executions through file handling.

The project also aimed to follow a modular programming approach. Instead of placing the entire implementation inside a single source file, individual components were separated into source and header files responsible for board rendering, snake behavior, food generation, keyboard input, score and game-data management, and console utilities. This organization was intended to make the program easier to understand, modify, test, and maintain.

B. Overall Program Architecture

The Classic Snake Game follows a modular architecture in which the `main.c` file acts as the central controller of the program. At the beginning of execution, the program loads previously stored high-score information, initializes the board, creates the initial snake, generates the first food item, and prepares the console display. The program then enters the main game loop, where input processing, snake movement, collision checking, food handling, speed adjustment, and screen rendering are repeatedly performed until the game terminates.

The implementation is divided into several modules. The `snake.c` and `snake.h` files contain the data and functions required for snake initialization, movement, growth, direction handling, and collision detection. The `food.c` and `food.h` files manage random food placement and score updates. The `input.c` and `input.h` files process keyboard input, while `board.c` and `board.h` maintain and display the two-dimensional game board. The `game.c` and `game.h` files manage persistent leaderboard data, and `utils.c` and `utils.h` provide Windows console-

related operations such as cursor control and text coloring.

This separation of responsibilities allows the main game loop to coordinate the different systems without containing the complete implementation of every individual feature.

The overall modular organization of the program is illustrated in Fig. 1.

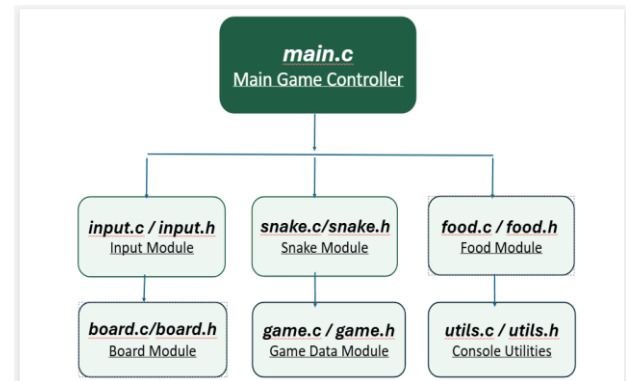


Fig. 1. Modular architecture of the Classic Snake Game.

III. GAME IMPLEMENTATION

A. Game Board Representation

The playing area of the Classic Snake Game is represented using a two-dimensional character array declared as `board` [BOARD_HEIGHT][BOARD_WIDTH]. Each position in this array corresponds to one cell of the game field. At the beginning of the program, the board is initialized with blank space characters, and during every iteration of the game loop it is cleared and reconstructed before being displayed again. This approach allows the positions of the snake, food, and other game elements to be updated continuously without permanently modifying the previous frame.

Different characters are used internally to represent individual game objects. The snake head is stored using the '@' character, the body segments are represented using 'O', and the tail is represented using 'o'. Food is stored in the board array using '*'. During rendering, these symbols are displayed using different console colors to improve readability, while the food symbol is visually rendered as a heart character. The program also draws the outer boundary

of the playing field separately using horizontal and vertical border characters.

The board-rendering module additionally displays information such as the control keys, current score, snake length, game status, and highest score. When the game ends, the same board structure is reused to display the final score and the three-position leaderboard. Separating the internal board representation from the rendering process makes it possible to update the game state first and then produce a complete visual frame for the player.

```
char board [BOARD_HEIGHT] [BOARD_WIDTH];

void initializeBoard(void)
{
    int row, col;

    for (row = 0; row < BOARD_HEIGHT; row++)
    {
        for (col = 0; col < BOARD_WIDTH; col++)
        {
            board[row][col] = ' ';
        }
    }
}
```

Listing 1. Initialization of the two-dimensional game board.

B. Snake Representation and Movement

The snake is represented using two parallel integer arrays, `snakeX[]` and `snakeY[]`, where the same index in both arrays represents the position of one snake segment. The `snakeX` array stores the horizontal coordinate of each segment, while `snakeY` stores the corresponding vertical coordinate. The variable `snakeLength` keeps track of the current number of segments. At the beginning of the game, the snake contains three segments and initially moves toward the right.

Movement is performed by shifting the coordinates of every body segment toward the position previously occupied by the segment in front of it. This update is carried out from the tail toward the head so that existing coordinate values are not overwritten before they are used. After all body segments have been shifted, the position of the head is modified according to the current movement direction.

A switch statement is used to determine how the head coordinates change. Moving right increases the head's X-coordinate, moving left decreases it, moving

upward decreases the Y-coordinate, and moving downward increases it. This coordinate-based method allows the snake to move one cell at a time while maintaining the relative positions of all its body segments.

When food is collected, the `growSnake()` function adds another segment at the position of the existing tail and increases `snakeLength`. During the following movement update, the newly added segment naturally becomes part of the moving snake.

A simplified portion of the snake movement algorithm is shown in Listing 2.

```
for (i = snakeLength - 1; i > 0; i--)
{
    snakeX[i] = snakeX[i - 1];
    snakeY[i] = snakeY[i - 1];
}

switch (direction)
{
    case RIGHT:
        snakeX[0] ++;
        break;

    case LEFT:
        snakeX[0] --;
        break;

    case UP:
        snakeY[0] --;
        break;

    case DOWN:
        snakeY[0] ++;
        break;
}
```

Listing 2. Coordinate-based snake movement algorithm

C. User Input and Direction Control

User input is processed in a separate module through the `processInput()` function. The program uses `_kbhit()` to determine whether a key has been pressed and `_getch()` to read the key immediately without requiring the user to press Enter. This allows the game to respond to movement commands while the main game loop continues to run.

The W, A, S, and D keys are used to move the snake upward, left, downward, and right respectively. Both uppercase and lowercase inputs are accepted. After a valid key is detected, the corresponding direction is passed to the `changeDirection()` function.

To prevent invalid movement, the program does not allow the snake to reverse directly into itself. For example, if the snake is moving to the right, a command to move immediately to the left is ignored. The same restriction is applied to the other opposite direction pairs. This prevents an instant self-collision caused by an 180-degree turn.

D. Food Generation and Snake Growth

Food is placed on the game board using randomly generated X and Y coordinates. The `initializeFood()` function generates coordinates within the valid board dimensions using the `rand ()` function and then checks whether the selected position overlaps any part of the snake. If an overlap is detected, another position is generated until a valid empty cell is found.

During each iteration of the game loop, the position of the snake's head is compared with the current food coordinates. When both positions are equal, the program determines that the food has been collected. The `growSnake()` function is then called, the player's score is increased by one, and a new food position is generated.

This procedure ensures that food does not appear inside the snake and that every successful food collection directly affects both the score and the length of the snake.

E. Collision Detection

Collision detection is used to determine whether the snake has reached an invalid position and whether the current game should end. The implementation handles two major collision types: collision with the boundaries of the game board and collision with the snake's own body.

A boundary collision occurs when the coordinates of the snake's head move outside the valid width or height of the board. The program checks whether the head's X-coordinate is less than zero or greater than or equal to `BOARD_WIDTH` and performs the equivalent check for the Y-coordinate using `BOARD_HEIGHT`. Boundary checking is performed during snake movement, while the `checkCollision()` function also verifies the current head position.

Self-collision is detected by comparing the coordinates of the snake's head with the coordinates of every remaining body segment. A loop begins from index 1, since index 0 represents the head itself. If both the X and Y coordinates of any body segment are equal to those of the head, the function reports a collision. When a collision is detected, the main game loop terminates, and the game-over sequence is displayed.

The main collision – checking logic is shown in listing 3.

```
int checkCollision(void)
{
    int i;
    if (snakeX[0] < 0 || snakeX[0] >= BOARD_WIDTH ||
        snakeY[0] < 0 || snakeY[0] >= BOARD_HEIGHT)
    {
        return 1;
    }
    for (i = 1; i < snakeLength; i++)
    {
        if (snakeX[0] == snakeX[i] &&
            snakeY[0] == snakeY[i])
        {
            return 1;
        }
    }
    return 0;
}
```

Listing 3. Wall and self-collision detection logic.

F. Dynamic Speed and Difficulty System

The difficulty of the game increases progressively as the player earns a higher score. This is implemented by changing the delay between consecutive game frames. The program uses the Windows `Sleep ()` function to pause the game loop for a specified number of milliseconds after each frame.

At lower scores, a larger delay is used, causing the snake to move more slowly. As the score reaches predefined thresholds, the delay is reduced, and the snake therefore moves faster. The game changes speed when the score reaches 5, 10, 15, 20, and 25 points. At the highest implemented level, the delay is reduced to 30 milliseconds.

This progressive speed adjustment increases the difficulty of the game without changing its fundamental rules. As the player's performance improves, faster movement requires quicker reactions and more careful direction control.

TABLE**I****SCORE-BASED MOVEMENT SPEED**

Score Range	Frame Delay
0 - 4	200 ms
5 - 9	140 ms
10 - 14	110 ms
15 - 19	80 ms
20 - 24	50 ms
25 and above	30 ms

G. High-Score and File Handling System

The game includes a persistent leaderboard that stores the three highest scores achieved by players. The leaderboard information is represented using a user-defined structure named `GameData`, which contains an integer array called `highscores[3]`. A global variable of this structure is shared between the game-data and rendering modules so that the stored scores can be accessed whenever they need to be displayed.

At the beginning of the program, the `loadGameData()` function attempts to open `data/highscores.txt` in read mode. If the file is available, the three stored score values are read into the `highscores` array. If the file cannot be opened, the scores remain initialized to zero, allowing the game to continue without terminating unexpectedly.

When a game ends, the current score is passed to `updateHighScores()`. The function compares the score with the existing first-, second-, and third-place values. If necessary, the older values are shifted downward so that the three scores remain in descending order. The updated leaderboard is then written back to the text file using `saveGameData()`.

This system allows leaderboard information to remain available even after the program has been closed and executed again. It also demonstrates the use of structures, arrays, file pointers, and standard C file-processing functions such as `fopen()`, `fscanf()`, `fprintf()`, and `fclose()`.

A simplified portion of the leaderboard update process is shown in Listing 4.

```
void updateHighScores(int currentScore)
{
    if (currentScore > gameData.highscores[0])
    {
        gameData.highscores[2] =
            gameData.highscores[1];
        gameData.highscores[1] =
            gameData.highscores[0];
        gameData.highscores[0] = currentScore;
    }
    else if (currentScore > gameData.highscores[1])
    {
        gameData.highscores[2] =
            gameData.highscores[1];
        gameData.highscores[1] = currentScore;
    }
    else if (currentScore > gameData.highscores[2])
    {
        gameData.highscores[2] = currentScore;
    }

    saveGameData();
}
```

Listing 4. Updating the three highest stored scores.

IV. TESTING AND RESULTS

A. Testing Procedure

The completed program was tested through repeated gameplay to verify the operation of its major features. Individual tests were performed for keyboard input, snake movement, food collection, snake growth, collision detection, score updating, speed progression, and persistent leaderboard storage. The objective of the testing process was to confirm that each component behaved correctly both independently and as part of the complete game loop.

Particular attention was given to gameplay situations that could cause incorrect behavior, including attempts to reverse the snake directly into itself, food generation on an occupied snake position, collisions with the game boundaries, self-collision, and the preservation of high-score information after the program was closed and executed again.

B. Test Cases and Results

TABLE II
FUNCTIONAL TEST CASES

Number	Test/Action	Expected Results	Status
1	Press W, A, S, and D during gameplay	Snake moves in the corresponding direction	Pass
2	Attempt an immediate 180° direction reversal	Opposite direction input is ignored	Pass
3	Snake collects food	Score increases by 1 and snake grows	Pass
4	Food is generated	Food appears at a position not occupied by the snake	Pass
5	Snake hits a boundary	Gameplay ends and Game Over is displayed	Pass
6	Snake head collides with its body	Gameplay ends	Pass
7	Score reaches higher thresholds	Snake movement becomes progressively faster	Pass
8	Achieve a leaderboard-worthy score and restart program	Updated high score remains stored	Pass



Fig. 2. Classic Snake Game during normal gameplay.



Fig. 3. Game-over screen displaying the final score and leaderboard.

C. Results and Discussion

The testing results indicate that the major functions of the Classic Snake Game operated as intended during repeated gameplay. The snake responded correctly to keyboard input, invalid 180-degree direction reversals were prevented, food collection increased both the score and snake length, and randomly generated food positions did not overlap the snake. Wall and self-collisions correctly terminated the game and displayed the game-over interface.

The progressive speed system also functioned according to the predefined score thresholds, making the game increasingly difficult as the player's score increased. In addition, the leaderboard system successfully retained the three highest scores by reading from and writing to the local high-score file. The gameplay screenshot in Fig. 2 demonstrates the normal running state of the program, while Fig. 3 shows the final game-over screen and stored leaderboard information.

Overall, the completed implementation successfully integrated the individual modules into a continuous game loop and produced the expected behavior for the tested features.

V. CHALLENGES, LIMITATIONS, AND SOLUTIONS

A. Maintaining Smooth Console Rendering

One important challenge in developing the game was continuously updating the console display without

allowing previous frames to remain visible on the screen. Since the snake moves automatically, the entire game board must be refreshed repeatedly during gameplay. The program addresses this by repositioning the console cursor to the top of the window and redrawing the complete board over the previous frame. This reduces visible flickering during continuous gameplay. The console cursor is also hidden during gameplay to produce a cleaner visual output.

B. Preventing Invalid Food Placement

Random food generation introduced another challenge because newly generated food could potentially appear in a position already occupied by the snake. To prevent this, the program generates a random coordinate and compares it with the coordinates of every current snake segment. If an overlap is detected, the position is rejected and another random coordinate is generated. This process continues until a valid unoccupied position is found.

C. Maintaining Correct Snake Movement

The movement of a multi-segment snake requires the position of every segment to be updated in the correct order. Updating the coordinates from the head toward the tail could overwrite position data that is still required by another segment. This problem is avoided by updating the snake from the tail toward the head. Each segment first copies the previous position of the segment directly in front of it, after which the head is moved according to the current direction.

The program also prevents immediate 180-degree direction reversals. Without this restriction, the snake could instantly move backward into its own body. The `changeDirection()` function therefore ignores commands that request the exact opposite of the current movement direction.

D. Preserving Leaderboard Data

A normal variable exists only while a program is running, so the high-score information would otherwise be lost whenever the game was closed. This was addressed using file handling. The three highest scores are stored in `data/highscores.txt`, loaded when the program starts, updated when a game ends,

and written back to the file. The leaderboard values are maintained using the `GameData` structure and a three-element score array.

E. Platform Dependency

The current limitation of the project is its dependence on Windows-specific console features. The program uses functions and headers such as `windows.h`, `conio.h`, `_kbhit()`, `_getch()`, `Sleep()`, and Windows console-control functions. As a result, the current implementation is primarily intended to run in a Windows console environment.

A future cross-platform version could replace these Windows-specific features with a portable terminal or graphics library. This would allow the game to run more consistently on operating systems such as Linux and macOS while preserving the existing gameplay logic.

VI. FUTURE IMPROVEMENTS

Although the current version of the Classic Snake Game provides the intended core gameplay features, several improvements could be introduced in future versions. One possible improvement is the addition of multiple difficulty levels, allowing players to choose an initial movement speed before starting the game. A pause and restart system could also be implemented to provide greater control during gameplay.

The game could further be expanded by introducing obstacles, different map layouts, bonus food items, and additional scoring mechanics. Player names could also be associated with the leaderboard so that the stored high scores identify individual players rather than displaying score values only.

Another significant improvement would be to reduce the program's dependence on Windows-specific console functions. Replacing the current input and rendering system with a cross-platform terminal or graphics library could allow the game to operate on additional operating systems. A graphical version of the game could also include improved visual elements, sound effects, menus, animations, and customizable themes while preserving the existing movement, collision, scoring, and leaderboard logic.

VII. CONCLUSION

The Classic Snake Game project successfully demonstrates how fundamental concepts of the C programming language can be combined to develop a complete interactive application. The final program integrates arrays, two-dimensional arrays, loops, conditional statements, functions, structures, random number generation, file handling, and modular programming to implement the major elements of the traditional Snake game.

The project was divided into separate source and header files responsible for snake behaviour, food generation, keyboard input, board rendering, game-data management, and console utilities. This modular structure allowed the individual components to be developed separately while being coordinated through the main game loop. The completed application supports responsive keyboard control, snake growth, random food generation, wall and self-collision detection, progressive difficulty, score tracking, and persistent storage of the three highest scores.

Developing the project provided practical experience in transforming individual programming concepts into a larger coordinated system. It also demonstrated the importance of program organization, testing, debugging, and separating different responsibilities when developing software beyond small introductory exercises.

ACKNOWLEDGMENT

The development of this project was supported by the course materials provided for CSE115, particularly Volumes 2–4, which were used as references for fundamental C programming concepts and implementation approaches. OpenAI's ChatGPT was also used as an AI-assisted learning and development tool for programming explanations, debugging guidance, code review, discussion of implementation approaches, and assistance during the preparation of project documentation. A YouTube video by Eight Sprites [3] was also consulted for general ideas and inspiration during the early development of the project. All final implementation, integration, modification,

testing, and project decisions were carried out by the student.

REFERENCES

- [1] CSE115 Course Materials, Vols. 2–4, North South University, course lecture materials, 2026.
- [2] OpenAI, “ChatGPT,” AI-assisted programming, debugging, and learning tool, accessed June 15 to August 17, 2026.
- [3] Eight Sprites, “Live Coding Snake Game in C.,” YouTube. [Online]. Available: https://youtu.be/t3y2b2_moY8. Accessed: June 21, 2026